ITSimplera Solutions

BLUE TEAM Internship

Week 1 Report

Task 1 : Deploy and Configure Wazuh Agent

Team Lead: Zia Ullah

Prepared By: Hafsa Murad

Company: ITSimplera Solutions

Report Period: Week 1 — June 29 – July 4, 2026

Table of Contents

Summary

This report documents the activities completed during Week 1 of the Blue Team Internship at ITSimplera Solutions, under the guidance of Team Lead Zia Ullah. The assigned task, Task 1: Deploy and Configure Wazuh Agent, required setting up a complete, working Wazuh Security Information and Event Management (SIEM) environment from the ground up and validating its core detection capabilities.

Over the course of the week, I deployed the Wazuh virtual appliance (manager, indexer, and dashboard) on an Ubuntu server, configured the Wazuh Manager, and installed Wazuh agents on both a Windows 10 endpoint and a Linux (Ubuntu) endpoint. Both agents were successfully registered with the central manager and appeared as active endpoints on the dashboard. I then configured and validated File Integrity Monitoring (FIM) by defining a monitored directory and confirming that file creation, modification, and deletion events were captured and correctly classified in real time.

To confirm the alerting pipeline end-to-end, I generated a series of controlled security events on the Windows endpoint — including user account creation/deletion, service stop/start, firewall state changes, and failed login attempts — and verified that each event was captured, parsed, and displayed as an alert on the Wazuh dashboard with the correct rule ID, severity level, and description.

By the end of Week 1, all seven requirements of Task 1 were completed and documented: the Wazuh appliance was deployed, the manager was configured, agents were installed on both operating systems, agents were registered, FIM was configured and validated, security events were generated and verified, and every step was documented with supporting screenshots. This report captures that process in detail and reflects the key technical skills developed during the week.

Learning Objectives

The objectives set out for this task, and achieved during Week 1, were to:

- Understand the architecture of Wazuh as a unified XDR/SIEM platform, including the roles of the Wazuh Manager, Wazuh Indexer, Wazuh Dashboard, and Wazuh Agents.
- Gain hands-on experience deploying a Wazuh virtual appliance on a Linux (Ubuntu) server, including installing prerequisite packages and running the official installation assistant.
- Learn how to configure and access the Wazuh Manager and Dashboard, including handling self-signed TLS certificates and using generated administrator credentials.
- Develop the ability to install and register Wazuh agents on multiple operating systems (Windows and Linux) using the dashboard's guided deployment wizard.
- Understand how agent enrollment and manager–agent communication works, and how to verify agent connectivity and status.
- Configure File Integrity Monitoring (FIM) to track file additions, modifications, and deletions on a monitored endpoint, and validate detection in real time.
- Generate realistic security events — authentication failures, account lifecycle changes, service state changes, and firewall changes — and trace them through to dashboard alerts.
- Practice interpreting SIEM dashboards, rule IDs, severity levels, and compliance/vulnerability widgets (PCI DSS mapping, MITRE ATT&CK, CIS benchmarks) as a foundation for SOC/blue-team analysis.
- Build professional technical documentation skills by recording each step of a security deployment with clear evidence and explanation.

1. Introduction

As part of the Blue Team Internship at ITSimplera Solutions, interns are assigned weekly, hands-on security operations tasks designed to build practical SOC and defensive-security skills. Week 1's assignment, Task 1, focused on Wazuh — a free and open-source security platform that unifies SIEM (Security Information and Event Management) and XDR (Extended Detection and Response) capabilities for endpoints and cloud workloads.

The objective of this task was to deploy a working Wazuh environment end-to-end: install and configure the Wazuh server components, connect endpoint agents from two different operating systems, enable File Integrity Monitoring, and prove that the platform correctly detects and alerts on security-relevant activity. The original task brief, as issued by the Blue Team lead, is reproduced below along with a summary of how each requirement was met.

1.1 Task Requirements

	Requirement	Report Section
✓	Deploy the Wazuh Virtual Appliance	*§2.1*
✓	Configure the Wazuh Manager	*§2.2*
✓	Install the Wazuh Agent on Windows and Linux	*§2.3 – §2.4*
✓	Register the agent with the Wazuh Manager	*§2.5*
✓	Configure and validate File Integrity Monitoring (FIM)	*§2.6*
✓	Generate security events and verify alerts	*§2.7*
✓	Document each step	*Full report*

2. Task 1: Deploy and Configure Wazuh Agent

This section documents, step by step, the complete deployment and configuration process — from provisioning the Wazuh server to verifying live security alerts.

2.1 Deploying the Wazuh Virtual Appliance

The Wazuh virtual appliance (manager, indexer, and dashboard, installed together in a single all-in-one deployment) was set up on a fresh Ubuntu server. This section covers environment preparation, dependency installation, and running the official Wazuh installation assistant.

2.1.1 Preparing the Ubuntu Environment

A new Ubuntu virtual machine was provisioned as the host for the Wazuh server. After the initial boot, the system was allowed to finish its first-time setup before proceeding with any package installation.

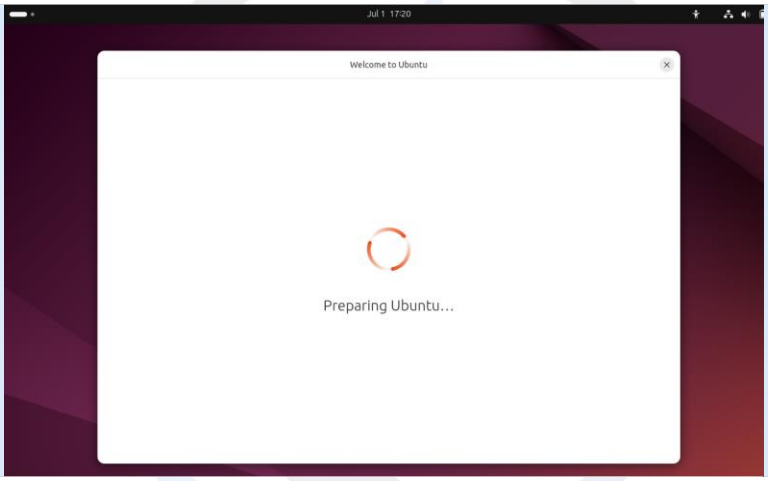


Figure 2.1 — Ubuntu preparing its environment on first boot.

2.1.2 Installing Required Packages

With Ubuntu ready, I switched to a root shell and updated the package index before installing the two prerequisites required for the Wazuh installer: curl (used to download the installation script) and default-jdk (a supporting dependency).

```
sudo su
sudo apt update
sudo apt install curl
sudo apt install default-jdk -y
```

```
hafsauser@ubuntu24:~$ sudo su
[sudo] password for hafsauser:
root@ubuntu24:/home/hafsauser# sudo apt update
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease

Hit:2 http://pk.archive.ubuntu.com/ubuntu noble InRelease

Hit:3 http://pk.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://pk.archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
69 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@ubuntu24:/home/hafsauser# sudo apt install curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  curl
0 upgraded, 1 newly installed, 0 to remove and 69 not upgraded.
Need to get 227 kB of archives.
After this operation, 535 kB of additional disk space will be used.
Get:1 http://pk.archive.ubuntu.com/ubuntu noble-updates/main amd64 curl
```

Figure 2.2 — Updating package lists and installing curl.

```
root@ubuntu24:/home/hafsauser# sudo apt install default-jdk -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  ca-certificates-java default-jdk-headless default-jre default-jre-head
less
  fonts-dejavu-extra java-common libatk-wrapper-java libatk-wrapper-java
-jni
  libice-dev libpthread-stubs0-dev libsm-dev libx11-dev libxau-dev libxc
b1-dev
  libxdmcp-dev libxt-dev openjdk-21-jdk openjdk-21-jdk-headless openjdk-
21-jre
  openjdk-21-jre-headless x11proto-dev xorg-sgml-doctools xtrans-dev
Suggested packages:
  libice-doc libsm-doc libx11-doc libxcb-doc libxt-doc openjdk-21-demo
  openjdk-21-source visualvm fonts-ipafont-gothic fonts-ipafont-mincho
  fonts-wqy-microhei | fonts-wqy-zenhei fonts-indic
The following NEW packages will be installed:
  ca-certificates-java default-jdk default-jdk-headless default-jre
  default-jre-headless fonts-dejavu-extra java-common libatk-wrapper-jav
a
```

Figure 2.3 — Installing default-jdk and its dependencies.

2.1.3 Downloading and Running the Wazuh Installation Assistant

I referred to the official Wazuh documentation portal to confirm the correct Quickstart installation command for the target version before running it on the server.

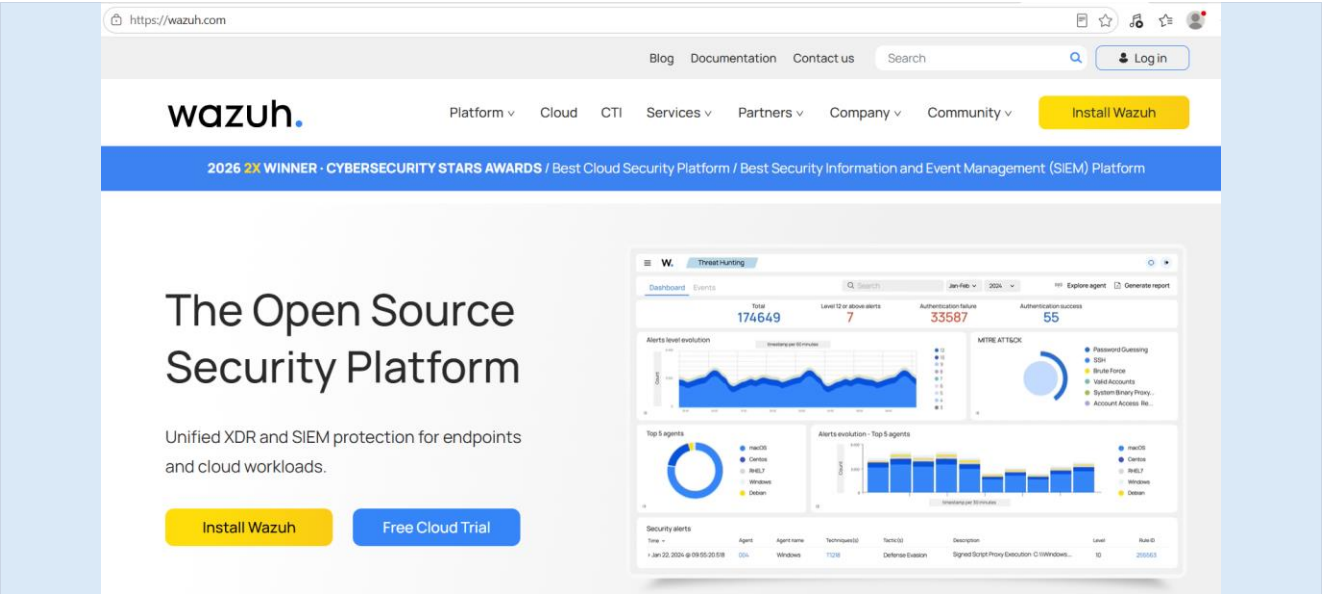


Figure 2.4 — Wazuh's official website (wazuh.com), the open-source security platform used for this deployment.

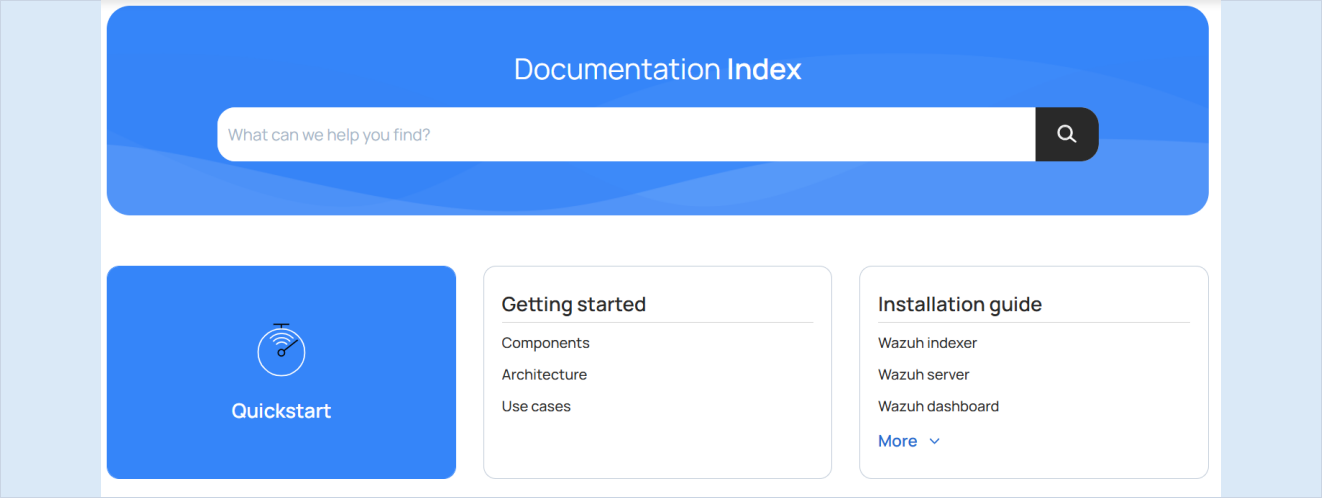


Figure 2.5 — Wazuh Documentation Index, used to navigate to the Quickstart installation guide.

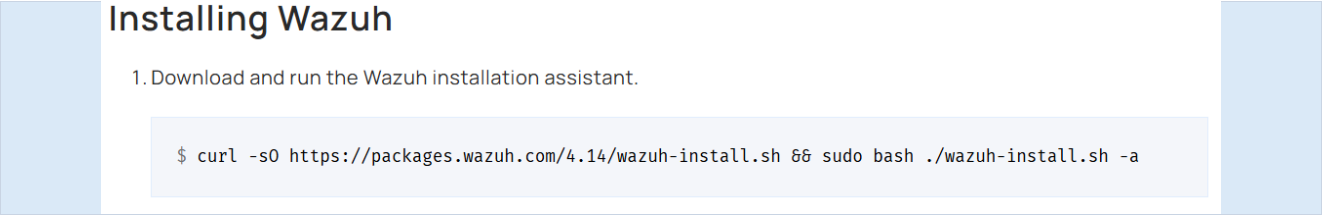


Figure 2.6 — The Quickstart page's single-command installation instruction for the Wazuh installation assistant.

The installation command downloads the wazuh-install.sh script and executes it with the -a flag, which performs an unattended, all-in-one installation of the Wazuh indexer, manager, and dashboard on the same host.

```
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a
```

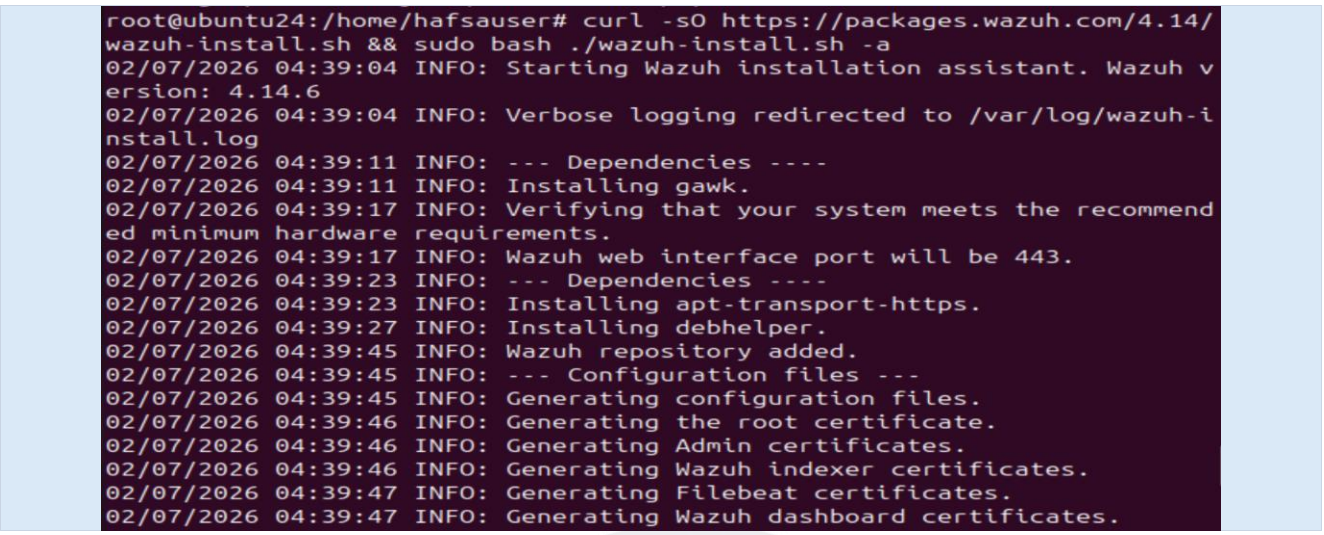


Figure 2.7 — Installation assistant starting: verifying system requirements, adding the Wazuh repository, and generating the root, admin, indexer, filebeat, and dashboard certificates.

The assistant continued through installing and starting each Wazuh component. On completion, it printed a summary containing the dashboard URL and the auto-generated administrator credentials, which I recorded securely. I then opened the manager's main configuration file (ossec.conf) to review the default configuration before moving on to agent deployment.

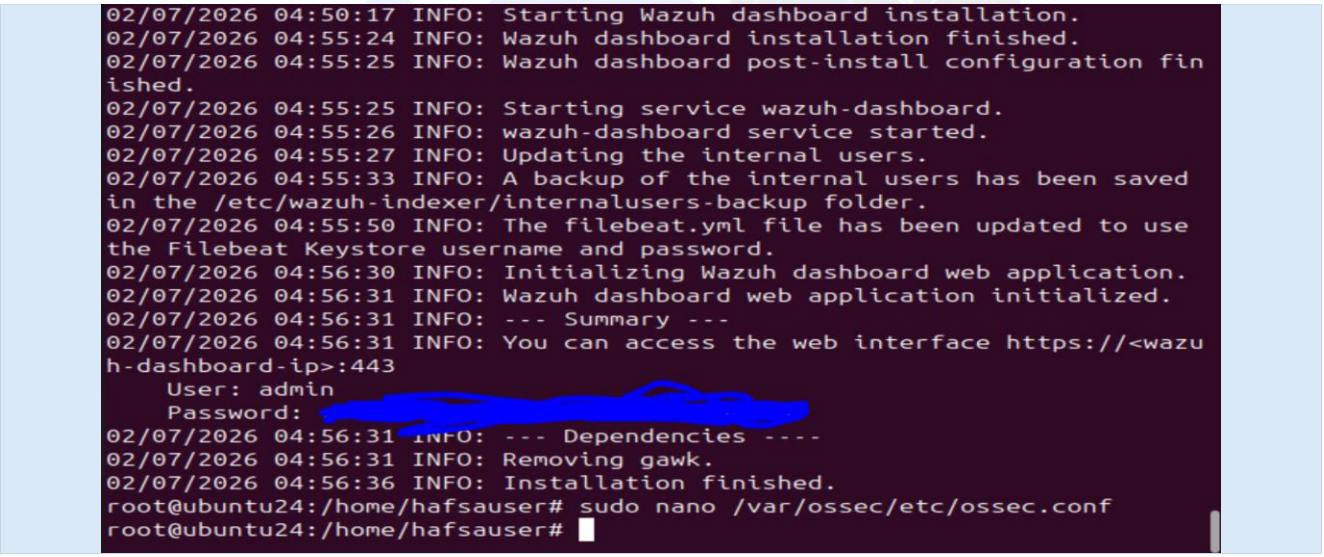


Figure 2.8 — Installation assistant completing: Wazuh dashboard service started, internal users updated, and the web interface URL and admin credentials generated. The final command opens ossec.conf for review.

> **Note**
> The generated administrator password is masked in this report for security reasons. In a production environment, this credential should be rotated and stored in a secrets manager rather than left at its installer-generated value.

2.2 Configuring the Wazuh Manager (Dashboard Access)

With the appliance installed, I accessed the Wazuh dashboard from a browser using the server's IP address on port 443. Because the dashboard uses a self-signed TLS certificate by default, the browser displayed a security warning; this was expected in a lab environment and was bypassed via the Advanced option to proceed to the site.

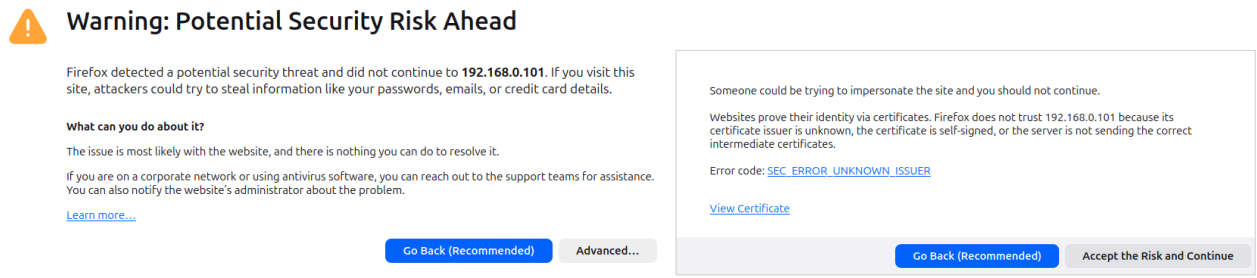


Figure 2.9 — Browser TLS warning caused by Wazuh's self-signed certificate; accepted to proceed to the dashboard.

I then logged in to the Wazuh dashboard using the admin username and the password generated during installation.

Figure 2.10 — Wazuh dashboard login screen.

On first login, the Overview page confirmed that the manager, indexer, and dashboard were all operational. Since no agents had been connected yet, the dashboard correctly reported zero registered agents while still displaying the available modules: Configuration Assessment, Malware Detection, File Integrity Monitoring, Threat Hunting, Vulnerability Detection, and MITRE ATT&CK.

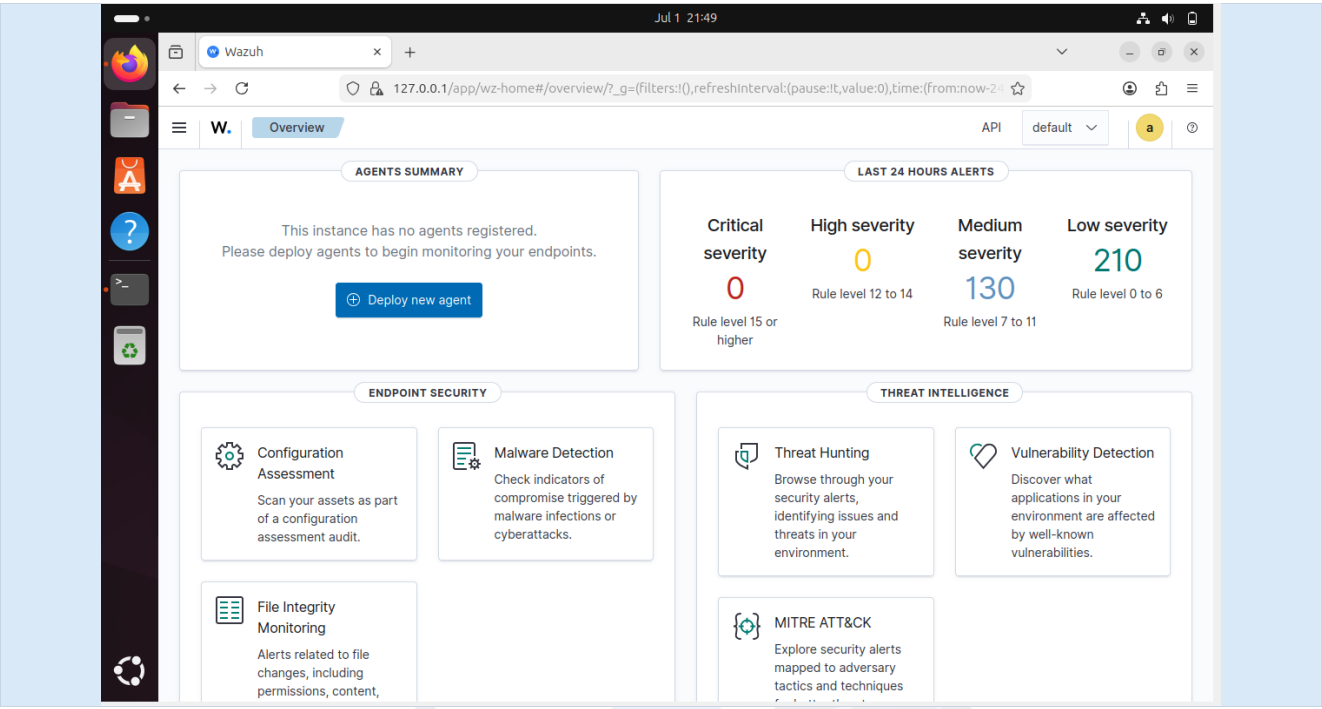


Figure 2.11 — Wazuh Overview page immediately after installation, confirming a healthy manager/indexer/dashboard stack with no agents registered yet.

2.3 Installing the Wazuh Agent — Windows

From the dashboard's Deploy new agent wizard, I selected Windows as the target operating system and MSI 32/64 bits as the package type.

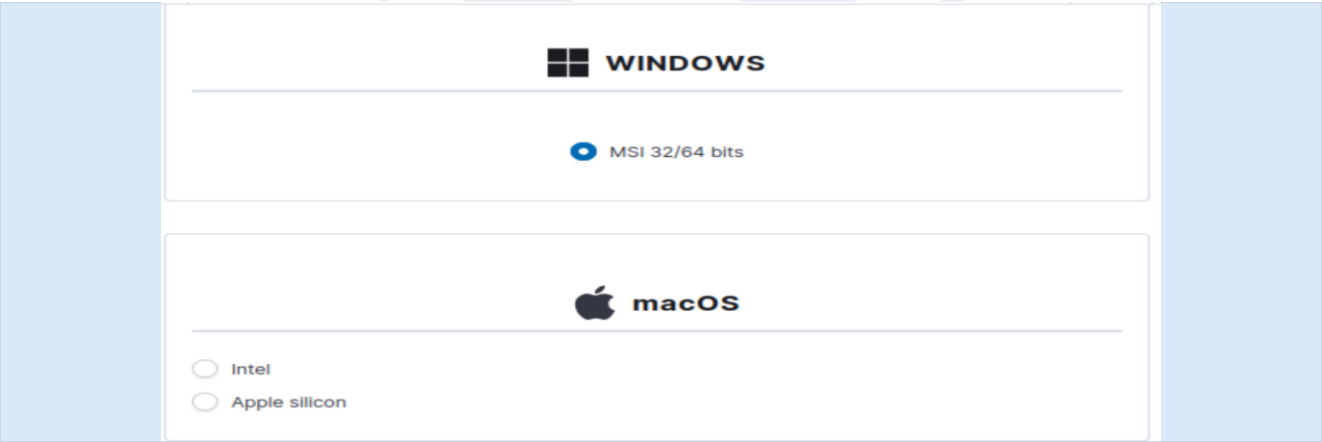


Figure 2.12 — Selecting the Windows MSI package in the Deploy new agent wizard.

Next, I supplied the manager's address so the agent would know where to send its data.

Figure 2.13 — Entering the Wazuh Manager's server address for the Windows agent.

I assigned a unique, easily identifiable agent name (SRV2026) instead of relying on the default hostname.

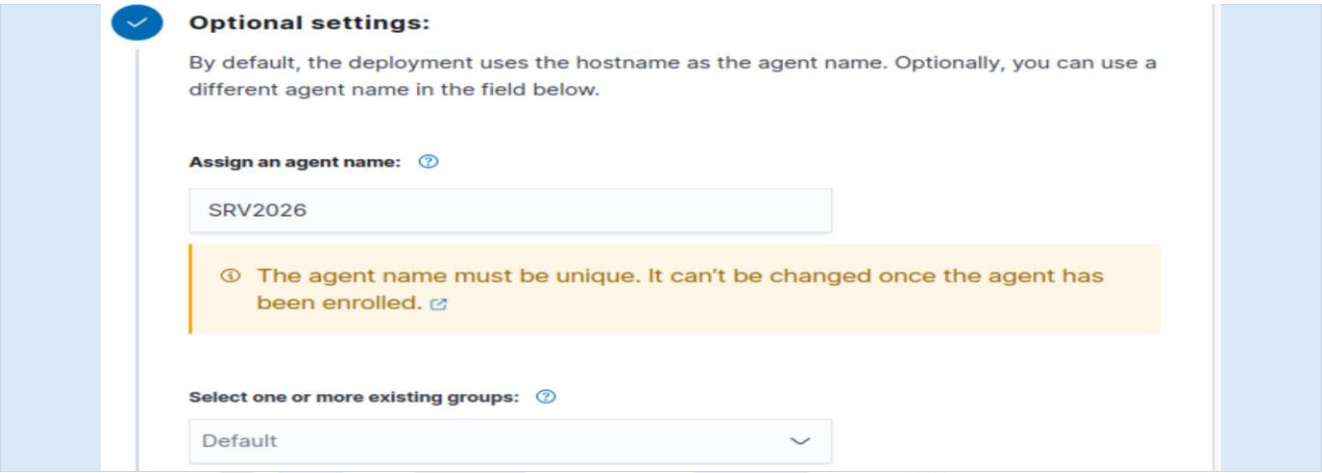


Figure 2.14 — Assigning a custom agent name and leaving the agent in the Default group.

The wizard then generated a ready-to-run PowerShell command, combining the download and silent installation (msiexec /q) of the agent with the manager address and agent name baked in as parameters.

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.6-1.msi
  -OutFile $env:tmp\wazuh-agent;
msiexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='127.0.0.1' WAZUH_AGENT_NAME='SRV2026'
```

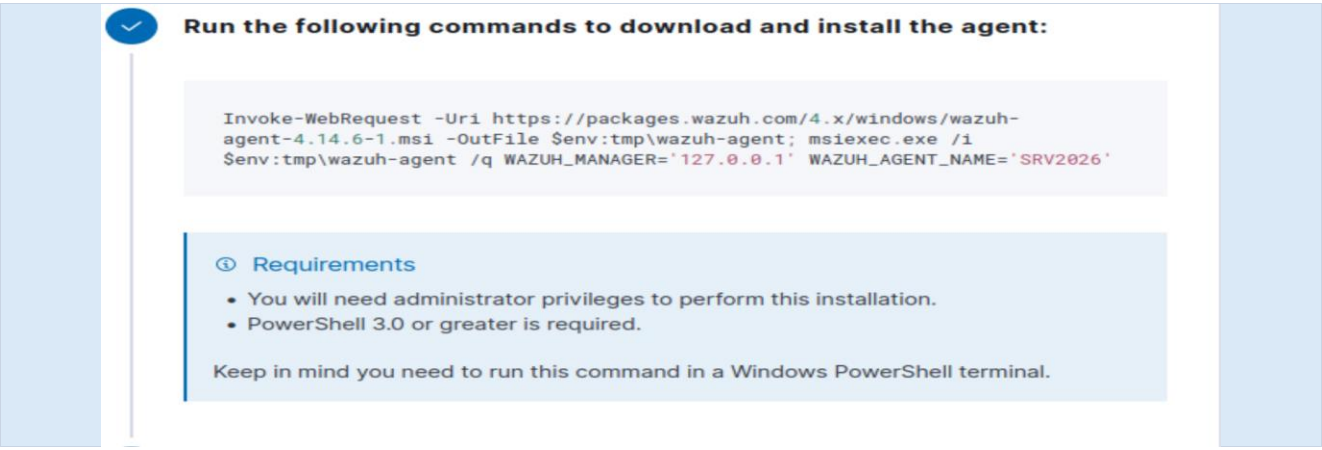


Figure 2.15 — Auto-generated PowerShell installation command with manager address and agent name pre-filled.

This command was run in an elevated Windows PowerShell terminal on the target Windows 10 machine, which requires administrator privileges and PowerShell 3.0 or later.

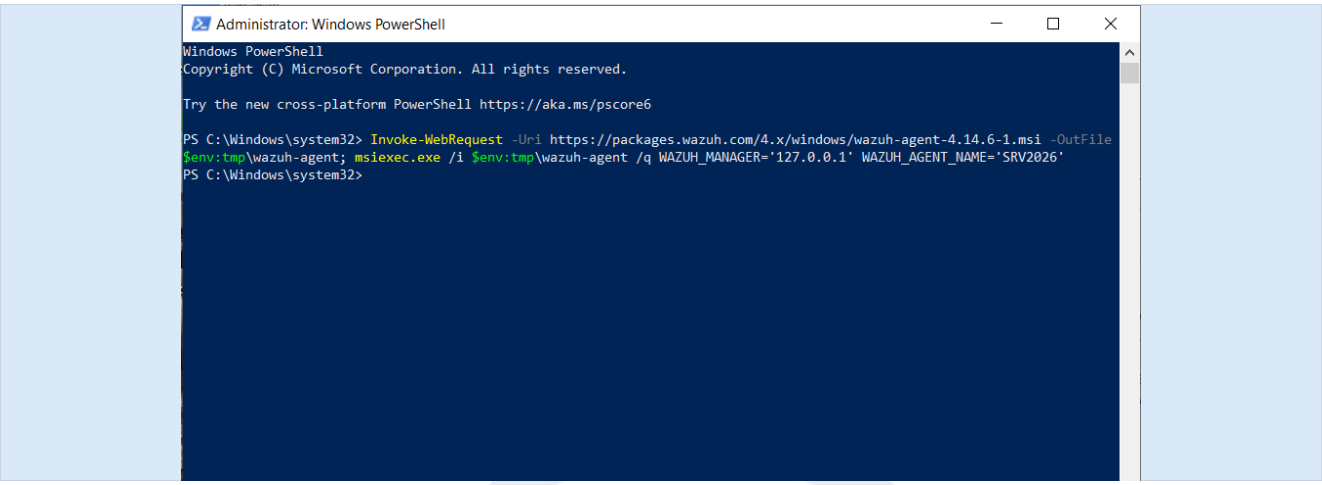


Figure 2.16 — Running the Wazuh agent installation command in an elevated PowerShell terminal.

After installation, I confirmed the Wazuh Agent was listed under installed applications in Windows' Programs and Features.

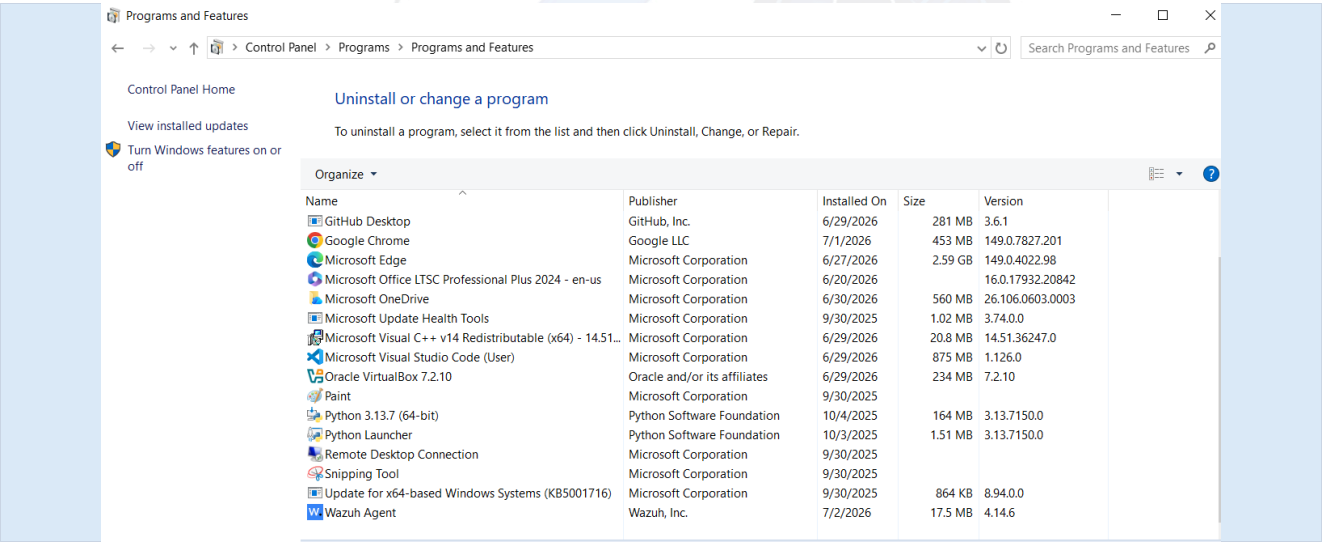


Figure 2.17 — Wazuh Agent listed in Programs and Features, version 4.14.6.

Finally, I verified the underlying Windows service (WazuhSvc) was set to start automatically and was in the Running state, confirming the agent was active.

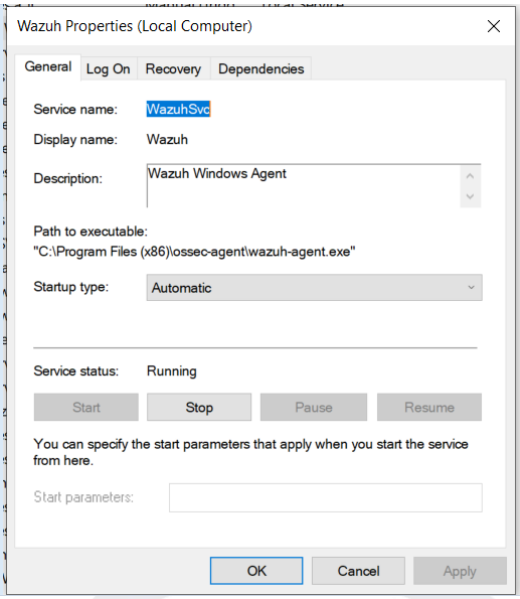


Figure 2.18 — Wazuh service properties: startup type Automatic, status Running.

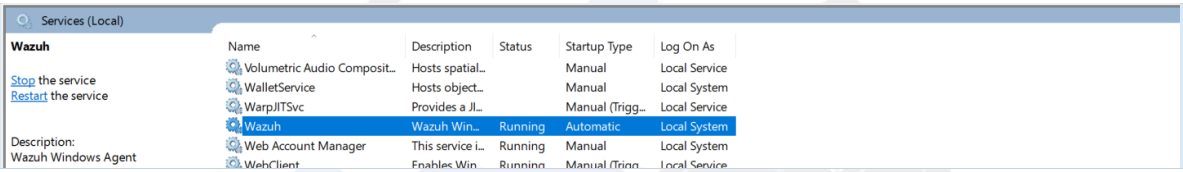


Figure 2.19 — Wazuh service confirmed Running under the Windows Services console.

2.4 Installing the Wazuh Agent — Linux

The same guided deployment process was used for the Linux endpoint. In the Deploy new agent wizard, I selected Linux and the DEB amd64 package (matching the target Ubuntu system), and entered the manager's address on the local network.

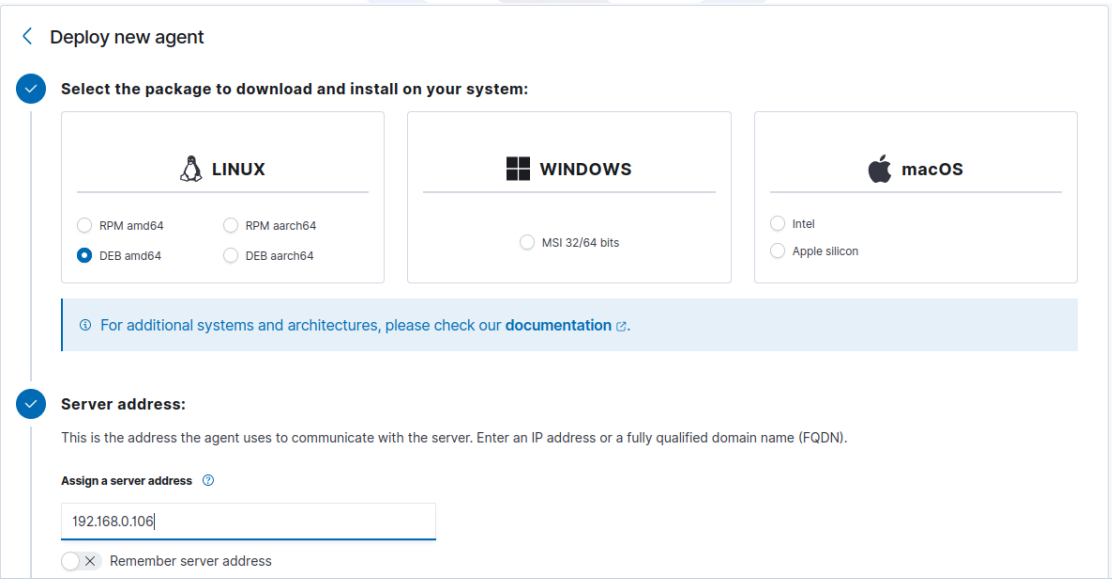


Figure 2.20 — Selecting the Linux DEB amd64 package and entering the Wazuh Manager server address (192.168.0.106).

As with the Windows agent, I assigned a descriptive, unique agent name — linux_agent — before generating the installation command, and kept the agent in the Default group.

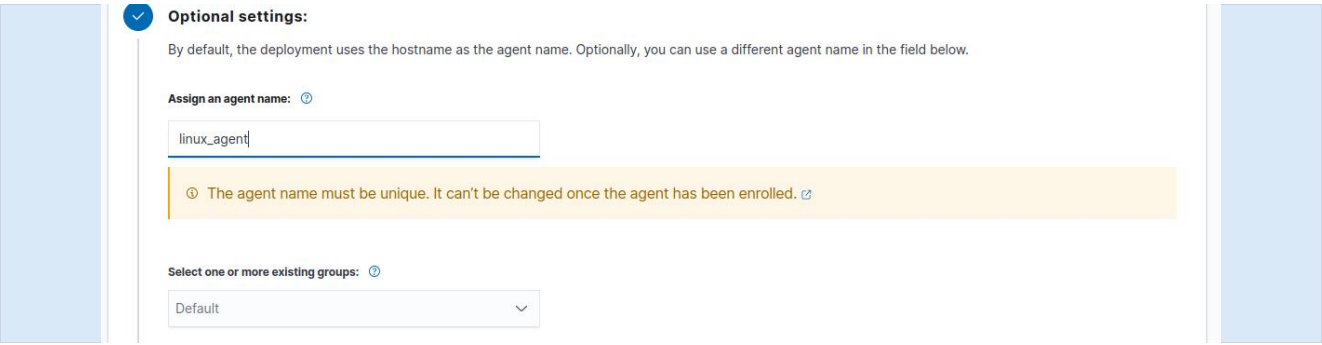


Figure 2.21 — Assigning the agent name linux_agent for the Linux endpoint.

The wizard produced the equivalent apt-based install and registration commands for Debian/Ubuntu, which were executed on the Linux endpoint's terminal in the same way as the Windows PowerShell command, downloading the .deb package, installing it, and writing the manager address and agent name into the agent's configuration before starting the wazuh-agent service.

Once installed, I confirmed the manager's REST API connection was healthy from the dashboard's Server APIs settings page — showing the default API entry online, pointed at host ubuntu24, running Wazuh v4.14.6 and up to date.

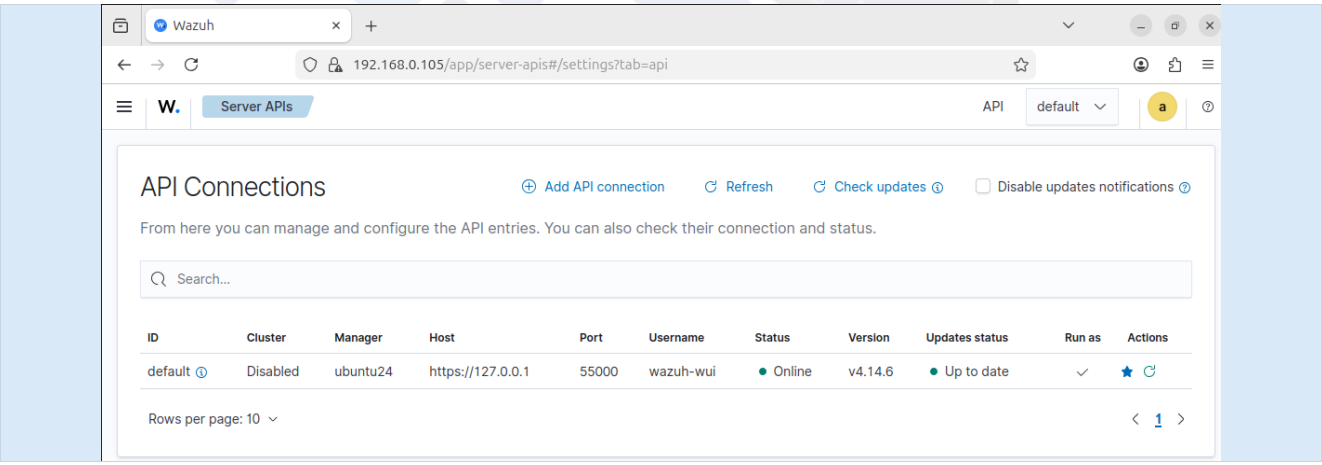


Figure 2.22 — Server APIs page confirming the default Wazuh Manager API connection is Online and up to date.

The Overview page then reflected the newly connected endpoint, with the Agents Summary donut updating to show one Active agent and zero disconnected agents.

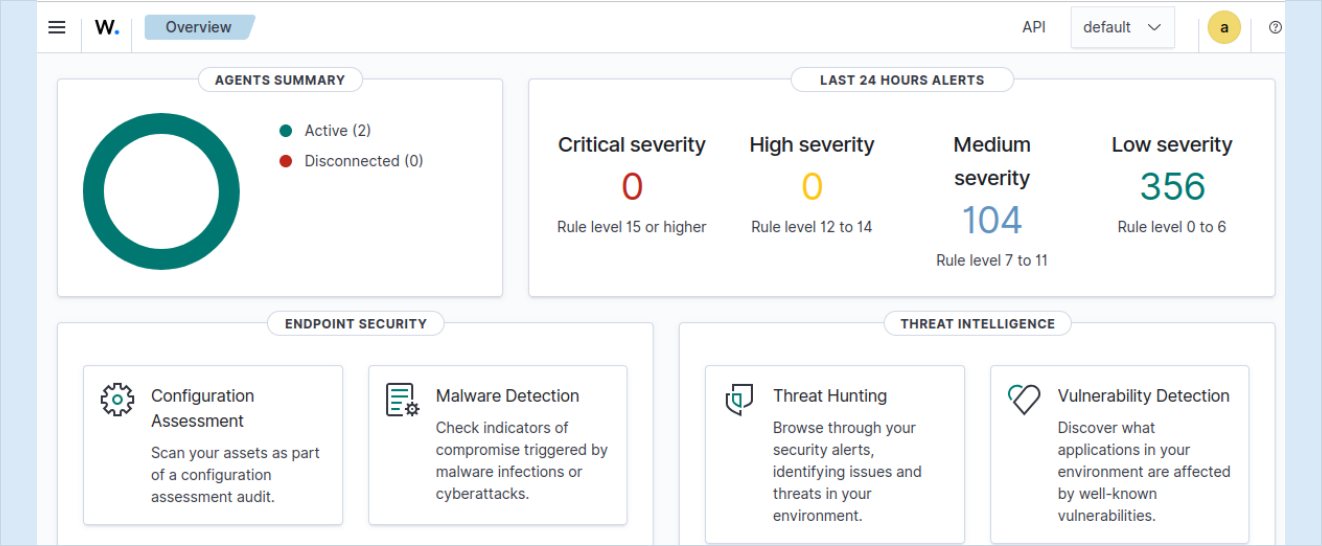


Figure 2.23 — Overview dashboard updates to show 1 Active agent after the Linux/Windows agent came online.

2.4.1 Deploying a Second Linux Endpoint — Linux Mint

To gain additional practice deploying the Wazuh agent on a second, independent Linux endpoint, a dedicated Linux Mint virtual machine was created in VirtualBox rather than reusing the existing Ubuntu manager VM. Installing an agent directly on the manager host was avoided, since both the Wazuh Manager and the Wazuh Agent packages install into the same /var/ossec/ directory, which would conflict with the manager's existing files; the manager also monitors itself automatically, so a separate local agent is unnecessary there. A new, lightweight VM was therefore the correct approach for a genuinely separate endpoint.

In the VirtualBox New Virtual Machine wizard, the VM was named and pointed at the Linux Mint 22.3 "Zena" (Cinnamon) ISO image, selected because it is the current long-term-support release (supported until April 2029) and is built on the same Ubuntu Noble package base used elsewhere in this deployment, keeping the Debian-family apt/dpkg commands consistent with the primary Linux agent.

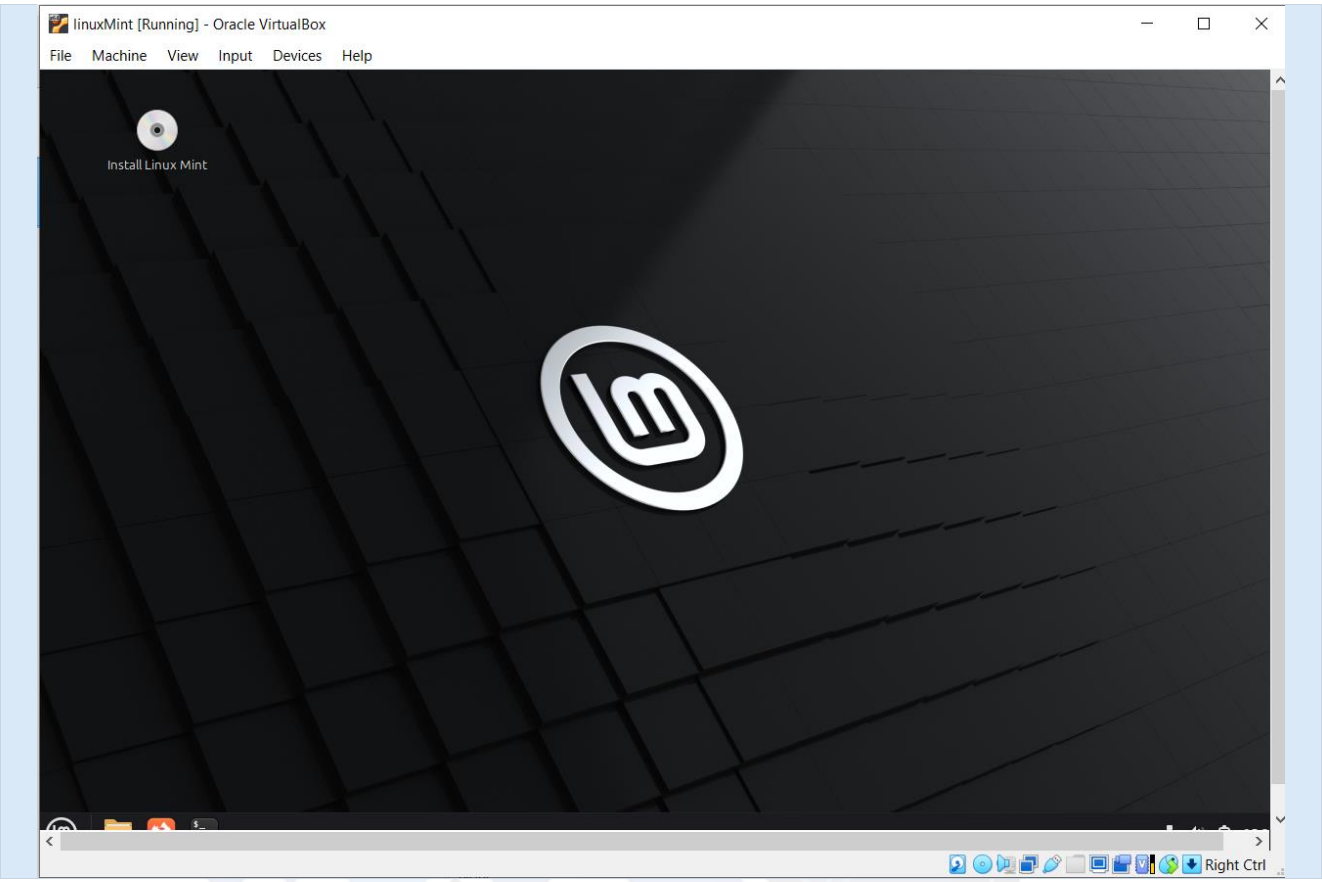


Figure 2.24 — VirtualBox New Virtual Machine wizard: unattended guest OS installation settings — user name, password, host name (agent), and Guest Additions ISO — for the Linux Mint agent VM.

Because the host machine's C: drive had very little free space remaining, the virtual hardware and virtual hard disk steps were reviewed carefully before finishing the wizard.

Figure 2.25 — Specifying virtual hardware for the agent VM: 2048 MB base memory and 1 CPU, deliberately kept small since a Wazuh agent is a lightweight background service compared with the 8 GB / 3-CPU manager VM.

A new 20 GB virtual hard disk (VDI, dynamically allocated) was created for the VM. The default save location was changed from the host's nearly-full C: drive to the E: drive, which had ample free space, avoiding a failed or crashed installation partway through.

Figure 2.26 — Virtual hard disk configuration: a new 20.49 GB VDI disk (ubuntu_agent.vdi) relocated to the E: drive rather than the nearly-full C: drive.

Once the VM finished its unattended installation and booted, the same Debian-family agent installation and manager-registration commands used for the primary Linux endpoint (Section 2.4) were applied — downloading the .deb package with curl, installing it with dpkg, enabling and starting the wazuh-agent service, and running agent-auth against the Wazuh Manager's address to complete enrollment. Because Linux Mint is built directly on Ubuntu, no changes to these commands were required.

```
# Debian/Ubuntu
curl -sO https://packages.wazuh.com/4.x/wazuh-agent-4.x.deb
sudo dpkg -i wazuh-agent-4.x.deb
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
# Register agent with manager
sudo /var/ossec/bin/agent-auth -m <Wazuh Manager IP>
sudo systemctl restart wazuh-agent
```

```
hafsauser@hafsauser-VirtualBox:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.6-1_amd64.deb
b
--2026-07-04 12:02:30--  https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.14.6-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 108.139.86.89, 108.139.86.29, 108.139.86.124, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|108.139.86.89|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 13220582 (13M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.14.6-1_amd64.deb'

wazuh-agent_4.14.6-1_amd64.deb                 15%[===================>
wazuh-agent_4.14.6-1_amd64.deb                100%[=================================================================

2026-07-04 12:06:10 (58.9 KB/s) - 'wazuh-agent_4.14.6-1_amd64.deb' saved [13220582/13220582]

[sudo] password for hafsauser:
Sorry, try again.
[sudo] password for hafsauser:
Selecting previously unselected package wazuh-agent.
(Reading database ... 461063 files and directories currently installed.)
Preparing to unpack .../wazuh-agent_4.14.6-1_amd64.deb ...
Unpacking wazuh-agent (4.14.6-1) ...
Setting up wazuh-agent (4.14.6-1) ...
hafsauser@hafsauser-VirtualBox:~$ sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
Created symlink /etc/systemd/system/multi-user.target.wants/wazuh-agent.service → /usr/lib/systemd/system/wazuh-agent.service.
hafsauser@hafsauser-VirtualBox:~$
```

Figure 2.27 — Linux agent installation and manager-registration commands (download, dpkg install, enable/start the service, agent-auth registration, and service restart), applied to the Linux Mint endpoint.

After registration, the new endpoint appeared alongside the existing Windows and Linux (Ubuntu) agents in the dashboard's Agents Summary and Endpoints ▸ Agents inventory, confirmed active in the same way as described in Section 2.5.

2.5 Registering Agents with the Wazuh Manager

Agent registration (enrollment) happens automatically as part of the Deploy new agent workflow: each installation command embeds the manager's address, which the agent uses to perform its initial authentication and key exchange with the manager on first start-up. No separate manual registration step was required, but I verified successful registration from several places on the dashboard.

The Agents Summary widgets confirmed two active agents — one Windows and one Linux Mint — in the Default group:

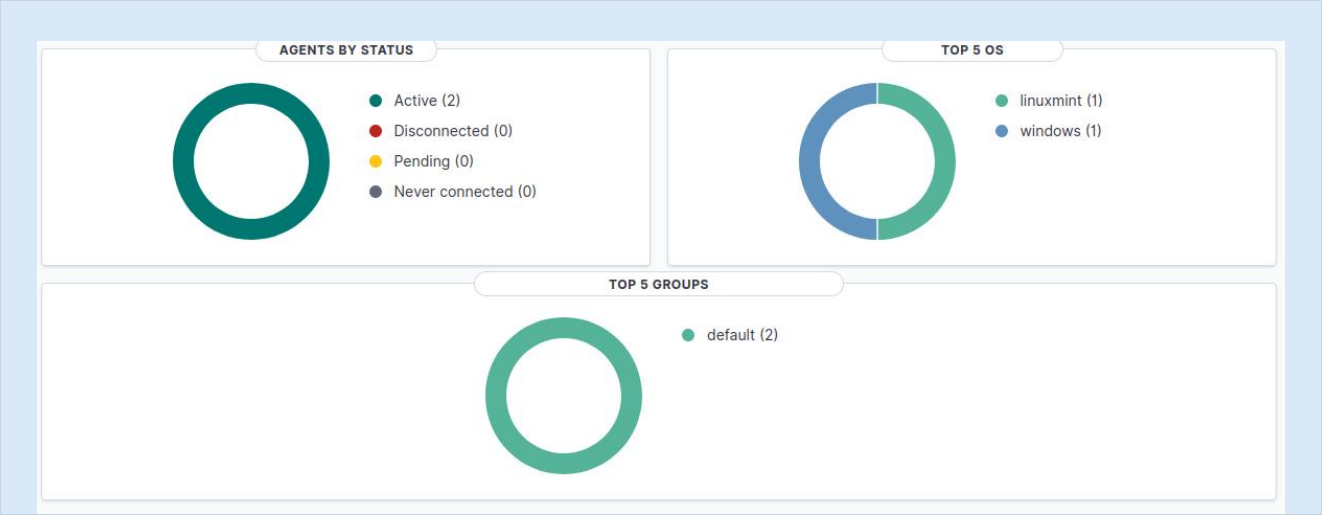


Figure 2.28 — Agents by Status, Top 5 OS, and Top 5 Groups widgets confirming two active agents (Windows and Linux Mint) in the Default group.

The full Endpoints ▸ Agents inventory listed both registered agents — 001 (SRV2026, Windows) and 002 (linux_agent, Linux Mint) — with their assigned IDs, names, IP addresses, group, detected operating system, cluster node, and Wazuh version, all showing an active status.

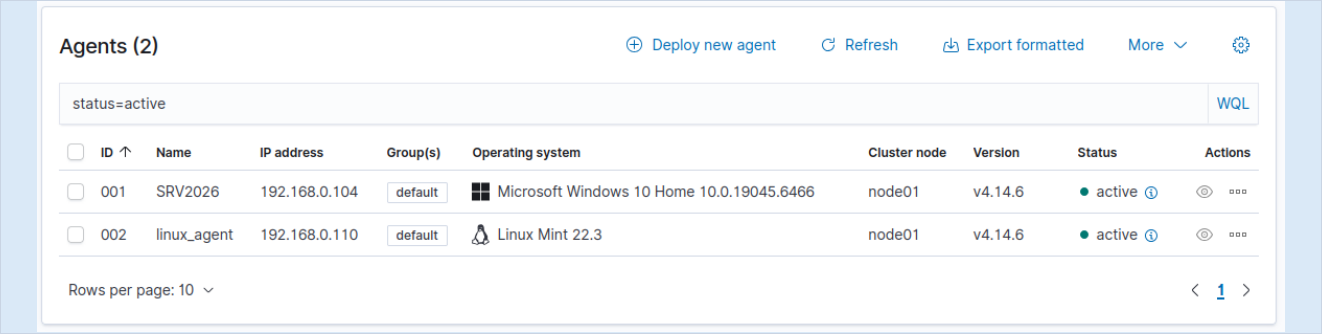


Figure 2.29 — Agents inventory table confirming both agents, 001 (SRV2026) and 002 (linux_agent), are registered and active on Wazuh v4.14.6.

Drilling into the individual agent's detail page provided a complete picture of the endpoint: hardware inventory (CPU, memory, host name, serial number), a live events-count graph, MITRE ATT&CK tactic mapping, PCI DSS compliance coverage, a vulnerability-detection summary, and a Security Configuration Assessment (SCA) score — confirming that the registered agent was actively reporting telemetry back to the manager.

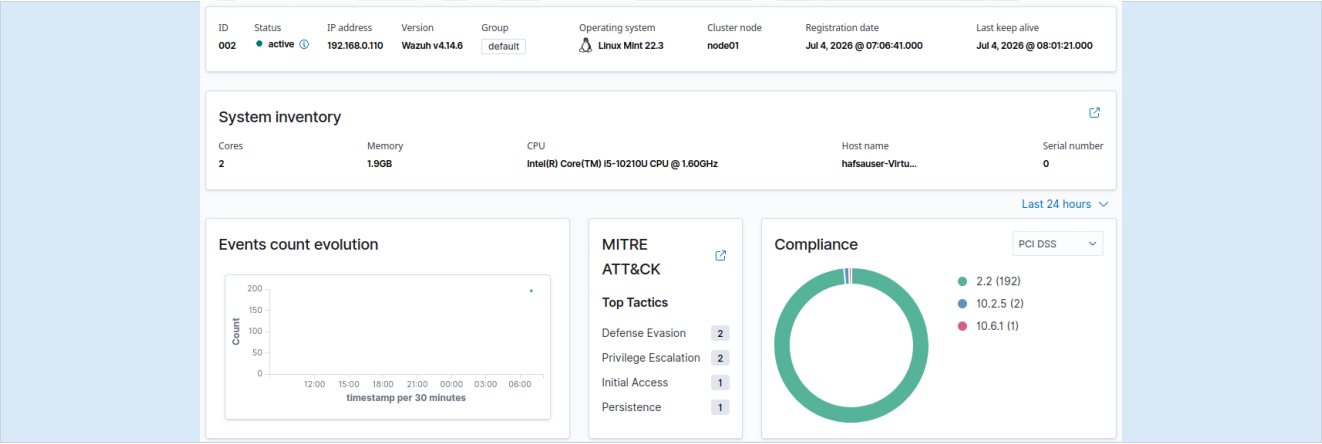


Figure 2.30 — Individual agent detail page (SRV2026) showing system inventory, events evolution, MITRE ATT&CK, compliance, vulnerability detection, and SCA summary widgets.

2.6 Configuring and Validating File Integrity Monitoring (FIM)

File Integrity Monitoring was enabled on the Windows endpoint by defining a monitored directory in the agent's syscheck configuration so that any file creation, modification, or deletion inside it would be captured and reported to the manager in real time. After updating the configuration, the Wazuh agent service was restarted to apply the new watch path.

To validate the configuration, I created a test file inside the monitored folder, modified it, and also created and then deleted a second file. Each action was picked up almost immediately and surfaced on the dashboard's FIM: Recent Events widget, correctly classified by action type (added, modified, deleted) with the matching rule ID and severity level.

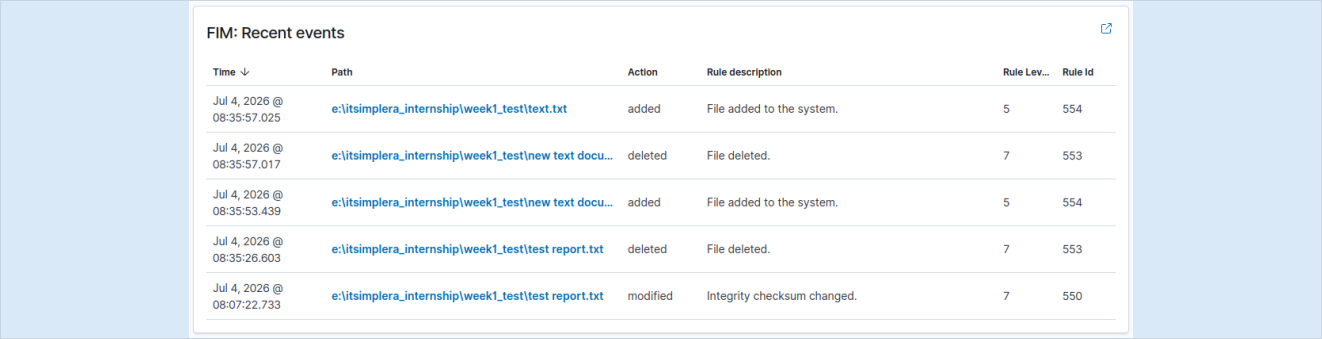


Figure 2.31 — FIM: Recent Events showing detected add, modify, and delete actions on files inside the monitored folder, with rule IDs 550/553/554.

The agent's overview page reflected the same activity alongside the endpoint's live inventory, events graph, MITRE ATT&CK tagging (Defense Evasion), and PCI DSS compliance breakdown — confirming FIM events were being correlated with the wider security context rather than treated in isolation.

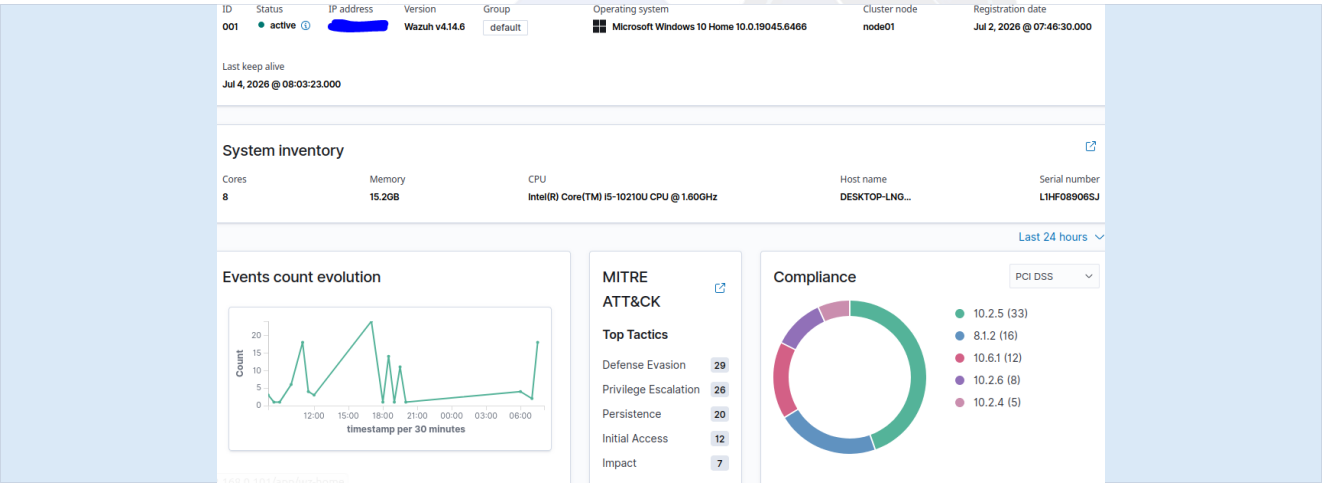


Figure 2.32 — Agent overview combining system inventory, events evolution, MITRE ATT&CK tactics, and compliance coverage alongside the FIM activity.

The same scan also surfaced Vulnerability Detection and Security Configuration Assessment results for the endpoint: several critical/high vulnerabilities were flagged against the installed Windows build and supporting packages, and a CIS Microsoft Windows 10 Enterprise Benchmark scan returned a baseline compliance score of 27% (117 passed / 302 failed / 5 not applicable out of 424 checks) — an expected result for a default, non-hardened lab machine rather than an error in the platform.

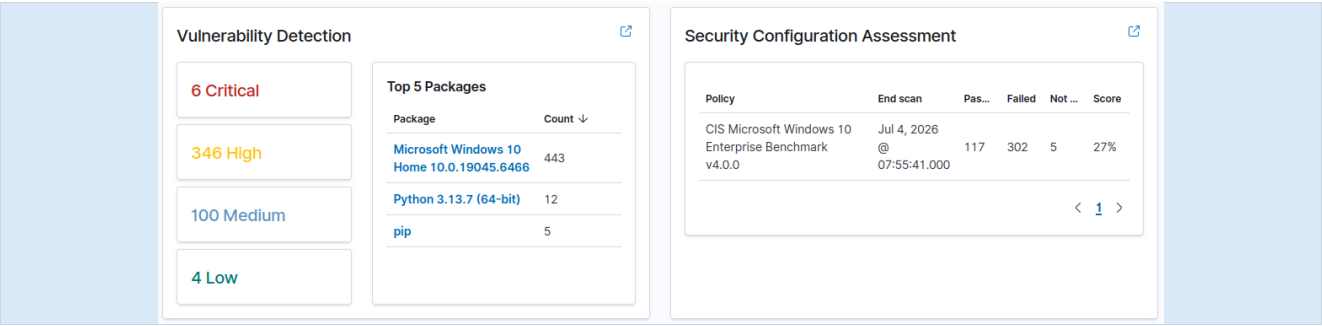


Figure 2.33 — Vulnerability Detection summary (6 Critical / 346 High / 100 Medium / 4 Low) and Security Configuration Assessment score for the endpoint.

Individual SCA checks can be inspected in detail — for example, check 15500 ("Ensure 'Enforce password...'") shows the exact PowerShell command Wazuh ran to evaluate the control and the resulting Failed status, which is useful for building a future remediation checklist.

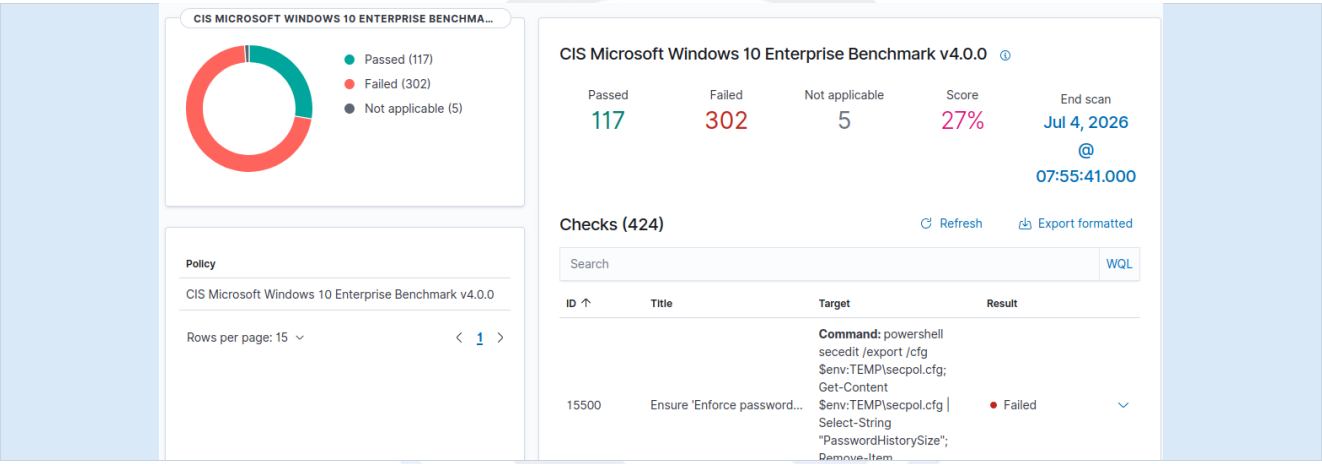


Figure 2.34 — Drilling into an individual CIS benchmark check, showing the evaluation command and Failed result.

Zooming out to the dedicated File Integrity Monitoring module, the alert volume for the monitoring window totalled 118 events, distributed mainly across the windows, windows_security, and windows_application rule groups, with a clear spike corresponding to the initial agent deployment and configuration scan.

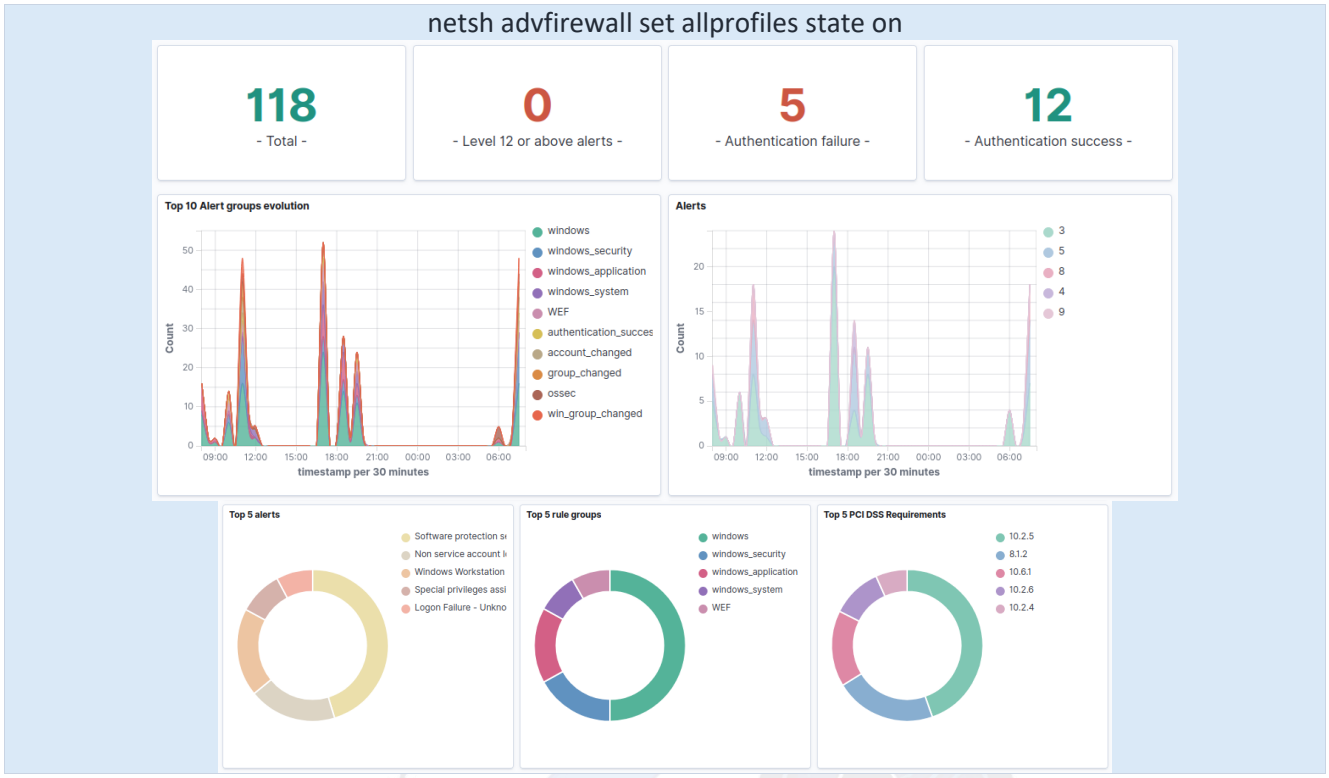


Figure 2.35 — FIM module summary: 118 total alerts, with Top 10 Alert groups and Alerts evolution over time.

The Top 5 alerts, Top 5 rule groups, and Top 5 PCI DSS requirements widgets provided a quick breakdown of what triggered most often — file-addition events, agent start-up events, and CIS benchmark findings — each mapped to the relevant PCI DSS control.

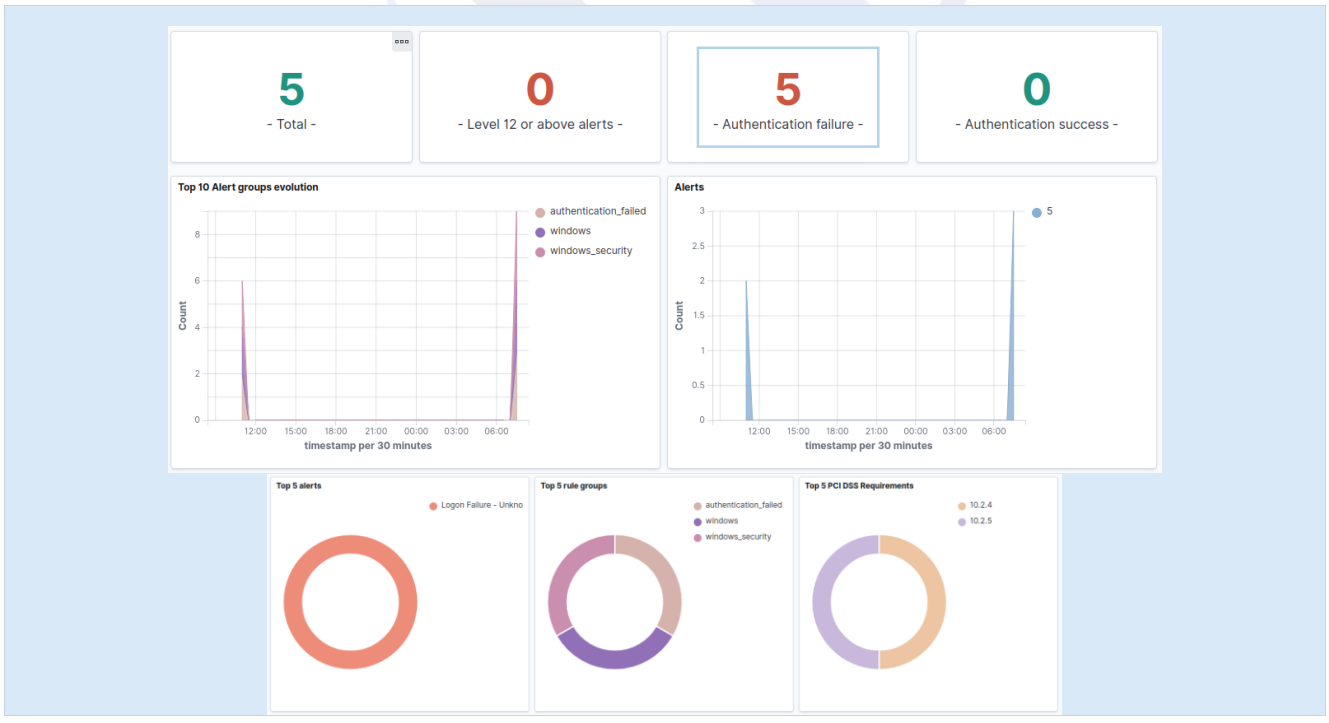


Figure 2.36 — Security Events filtered to the authentication failure alert group: 5 total alerts, all classified as "Logon Failure – Unknown user or bad password" under the windows/windows_security rule groups.

Finally, the FIM-specific analytics confirmed the expected action distribution from my test — files added (40%), modified (20%), and deleted (40%) — all attributed to the single monitored path used for testing, which matches exactly the sequence of test actions I performed.

Figure 2.37 — FIM analytics: most active users, action breakdown (added/modified/deleted), and files added/modified/deleted by path.

> **Result**
> File Integrity Monitoring was successfully configured and validated: file creation, modification, and deletion inside the monitored directory were detected in real time and correctly classified with the appropriate Wazuh rule IDs (550, 553, 554).

2.7 Generating Security Events and Verifying Alerts

To validate the alerting pipeline end-to-end — from raw Windows event log entries, through the agent and manager, to a dashboard alert — I generated a controlled set of security-relevant actions on the Windows endpoint from an elevated Command Prompt:

- User account lifecycle: created a local test account (intern123) and then deleted it, to trigger account-created and account-deleted events.
- Service state change: stopped and then restarted the Wazuh service itself, to confirm service-status changes are logged.
- Firewall configuration change: disabled and then re-enabled the Windows Defender Firewall across all profiles.
- Failed login attempts: intentionally triggered failed Windows logon attempts to generate authentication-failure events.

```
net user intern123 P@ssword123 /add
net user intern123 /delete
net stop Wazuh
```

```
net start Wazuh
netsh advfirewall set allprofiles state off
netsh advfirewall set allprofiles state on
```

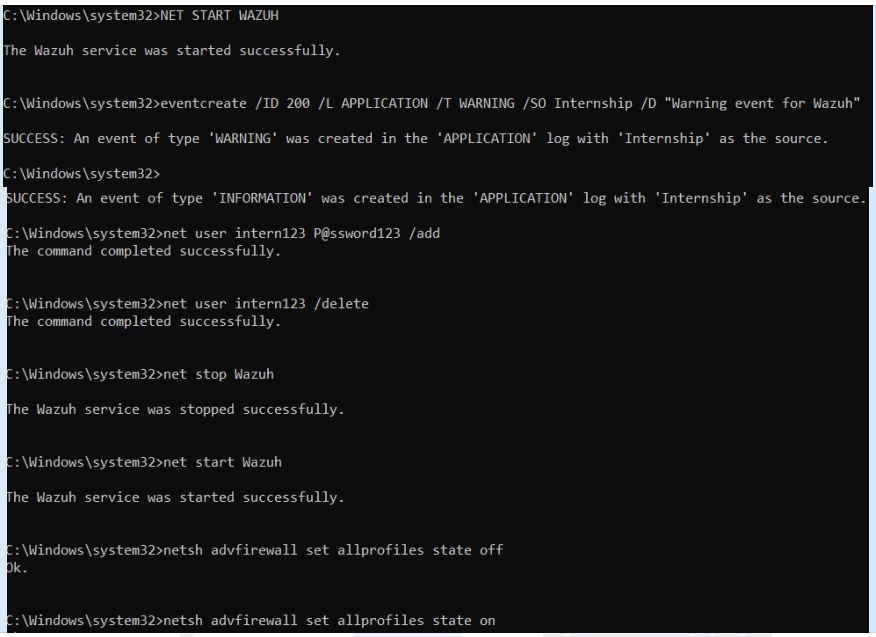


Figure 2.38 — Command Prompt output for the account-lifecycle, service-restart, and firewall-toggle test actions.

Each of these actions produced a corresponding alert on the Wazuh dashboard's Security Events module. During the test window, 12 total alerts were generated, including 0 authentication failures and 12 authentication successes, with zero critical (Level 12+) alerts — consistent with a controlled test rather than a real attack.

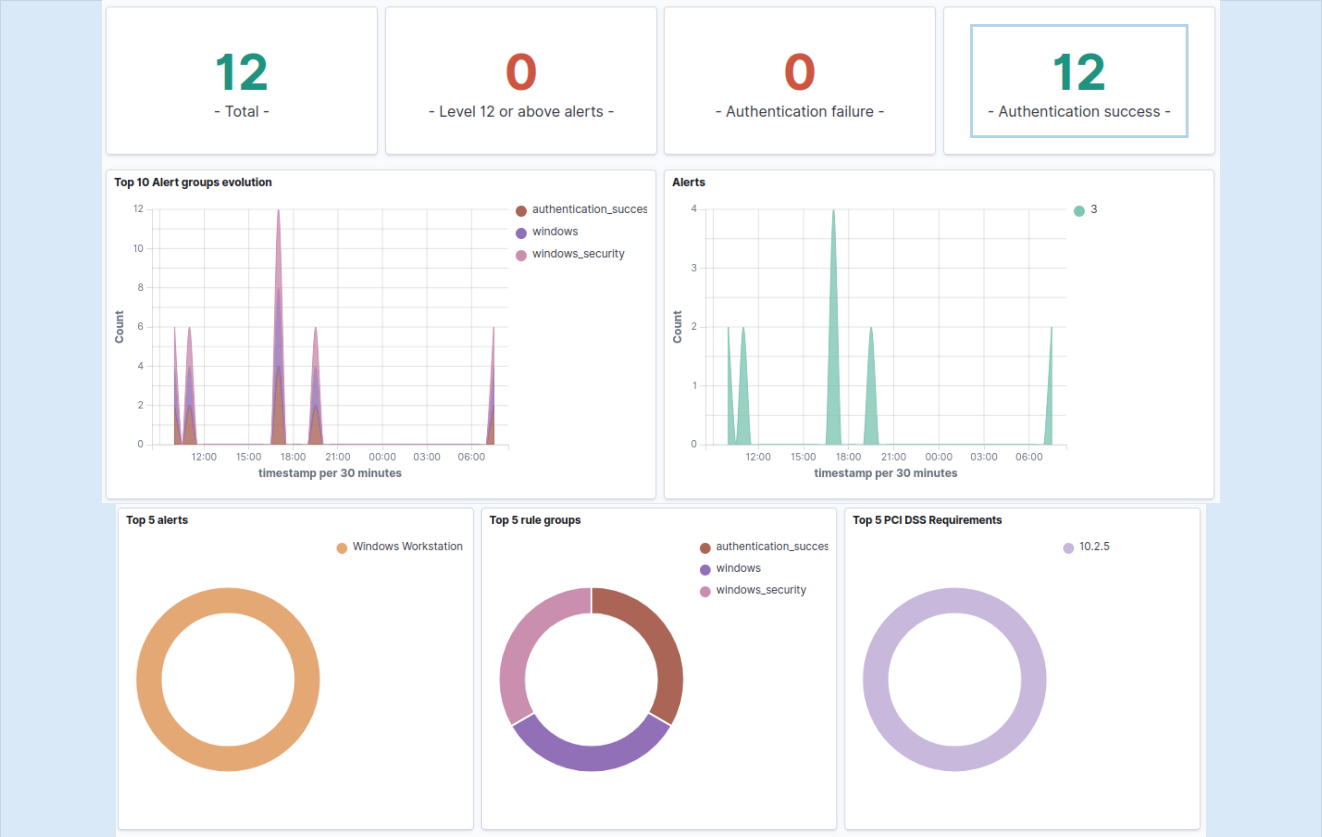


Figure 2.39 — Security Events summary: 12 total alerts, 0 authentication failures, 12 authentication successes, with the Top 10 Alert groups evolution over time.

Breaking the alerts down further, the Top 5 alerts, Top 5 rule groups, and Top 5 PCI DSS requirements widgets showed the events were correctly categorised under Windows, Windows Security, and Windows Application log sources, and mapped to relevant PCI DSS logging/monitoring requirements (10.2.5, 8.1.2, 10.6.1, 10.2.6, 11.5).

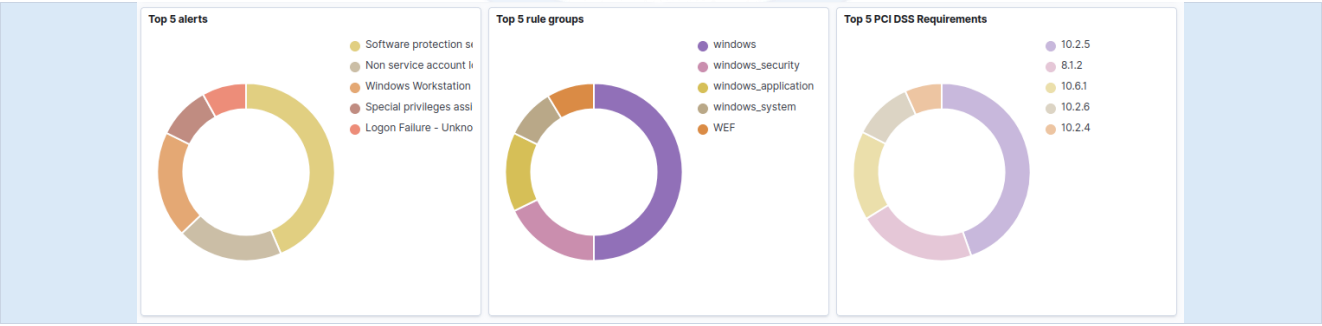
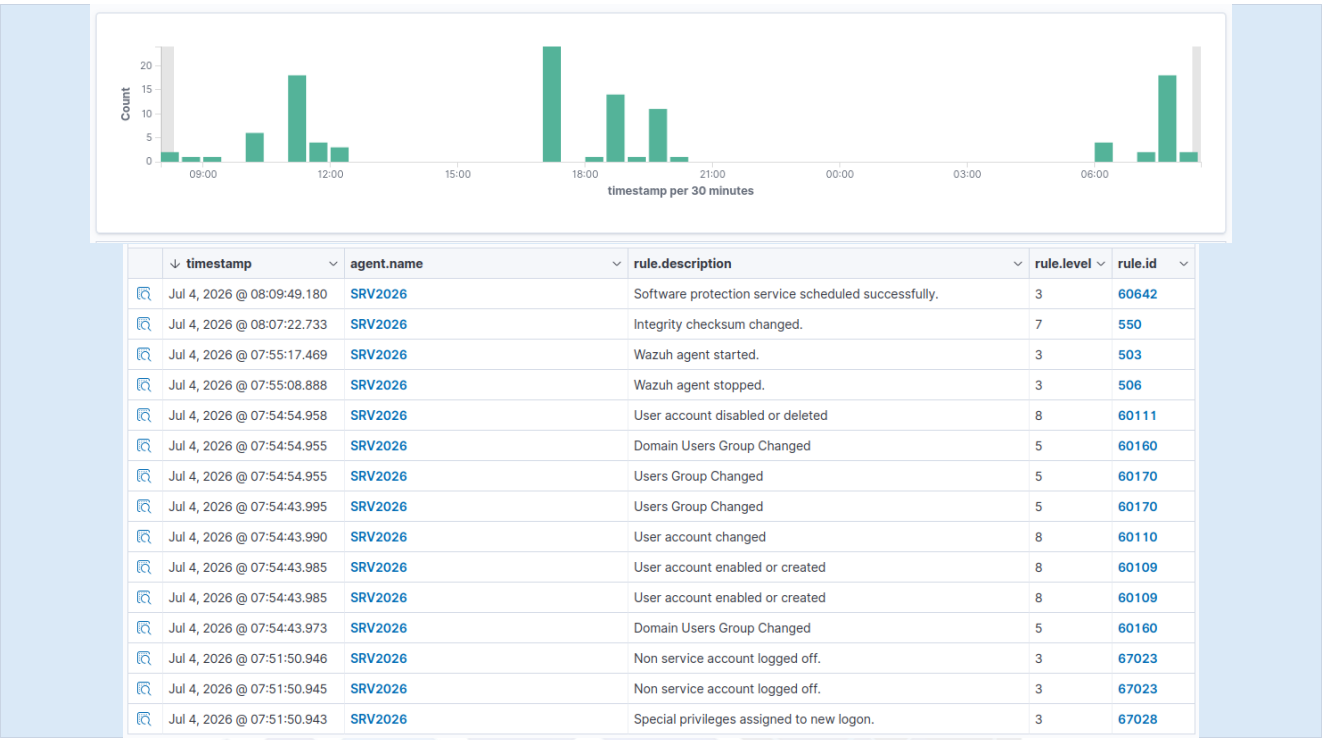


Figure 2.40 — Top 5 alerts, rule groups, and PCI DSS requirements generated by the test actions.

Finally, I opened the Security Events explorer for the Windows agent (SRV2026) to review the full timeline of alerts generated during testing. The table showed the expected mix of activity from each test action: user account creation/deletion and group membership changes (rule IDs 60109, 60110, 60111, 60160, 60170), the Wazuh service stopping and restarting (rule IDs 506 and 503), a file integrity checksum change (rule ID 550), a scheduled software-protection-service event (rule ID 60642), and logon-related events (rule IDs 67023 and

67028) — confirming that Wazuh correctly parsed and classified each of the different Windows Security and System event types produced by my test actions.



timestamp	agent.name	rule.description	rule.level	rule.id
Jul 4, 2026 @ 08:09:49.180	SRV2026	Software protection service scheduled successfully.	3	60642
Jul 4, 2026 @ 08:07:22.733	SRV2026	Integrity checksum changed.	7	550
Jul 4, 2026 @ 07:55:17.469	SRV2026	Wazuh agent started.	3	503
Jul 4, 2026 @ 07:55:08.888	SRV2026	Wazuh agent stopped.	3	506
Jul 4, 2026 @ 07:54:54.958	SRV2026	User account disabled or deleted	8	60111
Jul 4, 2026 @ 07:54:54.955	SRV2026	Domain Users Group Changed	5	60160
Jul 4, 2026 @ 07:54:54.955	SRV2026	Users Group Changed	5	60170
Jul 4, 2026 @ 07:54:43.995	SRV2026	Users Group Changed	5	60170
Jul 4, 2026 @ 07:54:43.990	SRV2026	User account changed	8	60110
Jul 4, 2026 @ 07:54:43.985	SRV2026	User account enabled or created	8	60109
Jul 4, 2026 @ 07:54:43.985	SRV2026	User account enabled or created	8	60109
Jul 4, 2026 @ 07:54:43.973	SRV2026	Domain Users Group Changed	5	60160
Jul 4, 2026 @ 07:51:50.946	SRV2026	Non service account logged off.	3	67023
Jul 4, 2026 @ 07:51:50.945	SRV2026	Non service account logged off.	3	67023
Jul 4, 2026 @ 07:51:50.943	SRV2026	Special privileges assigned to new logon.	3	67028

Figure 2.41 — Security Events view showing the range of alerts (account/group changes, service start/stop, integrity checksum, logon events) generated during testing.

Result

The full detection pipeline — agent log collection, manager rule/decoder matching, indexer storage, and dashboard alerting — was confirmed working correctly for account-management, service-state, firewall, and authentication events.

3. Challenges Faced

- Self-signed certificate warnings: the browser flagged the dashboard's default TLS certificate as untrusted on first access. This was resolved by explicitly accepting the certificate exception, which is expected behaviour for an internal/lab Wazuh deployment that has not yet been issued a certificate from a trusted CA.
- Low initial compliance score: the CIS benchmark scan returned only a 27% pass rate on the Windows endpoint. This was expected, since the machine was running default, non-hardened settings, and it was treated as a useful finding rather than a fault — it highlights follow-up hardening work for a future task.
- Understanding agent enrollment: initially it was not obvious how an agent authenticates with the manager without a separate manual registration step. This was clarified by using the dashboard's Deploy new agent wizard consistently, which automatically embeds the correct manager address and generates a matching install/enrollment command for each operating system.
- Attempting to install the agent on the manager host: an early attempt was made to install the Wazuh Agent package directly on the existing Ubuntu manager VM. This failed because both the manager and the agent install into the same /var/ossec/ directory, causing a package conflict. This was resolved by recognising that the manager already monitors itself automatically, and by provisioning a separate, dedicated VM (Linux Mint) for the second Linux endpoint instead.
- Host disk space constraints: the host machine's C: drive had only a few gigabytes of free space remaining, which was too little for a new 20 GB virtual disk and risked crashing the installation partway through. This was resolved by relocating the new VM's virtual hard disk (and the VirtualBox default machine folder and Windows Downloads folder going forward) to the E: drive, which had ample free space.

4. Conclusion

All seven requirements of Task 1 were completed during Week 1 of the Blue Team Internship: the Wazuh virtual appliance was deployed on Ubuntu, the Wazuh Manager was configured and accessed via the dashboard, agents were installed and registered on both a Windows and a Linux endpoint, File Integrity Monitoring was configured and validated with real file-change events, and a set of controlled security events was generated and successfully traced through to correctly classified dashboard alerts. Every step in this process has been documented above with supporting screenshots.

This task provided hands-on experience across several core blue-team skills: Linux server administration, Windows endpoint administration, SIEM/XDR deployment and configuration, log-based detection concepts, and the interpretation of SOC dashboards, rule IDs, and compliance widgets. It has also reinforced the importance of clear technical documentation when recording security operations work. This foundation will support the more advanced detection, tuning, and hardening tasks planned for the coming weeks of the internship.